INT428: AI - ESSENTIALS
PROJECT REPORT

(Project Semester January-May 2026)

AI Digital Asset Manager (AI-DAM)

Submitted by :

Name: Sumit Yadav

Registration No: 12412277

Programme and Section: B.Tech 424IM

Course Code: INT428

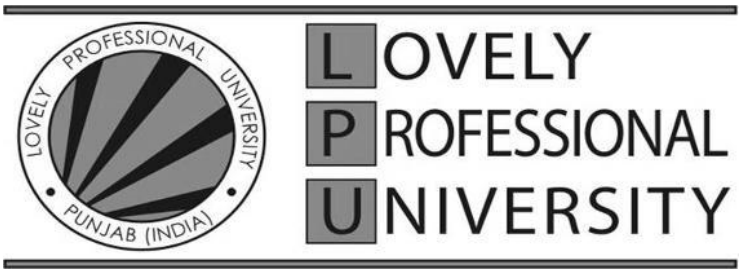
Under the Guidance of

Tarun Singh (34442)

Discipline of CSE/IT

Lovely School of AI & Emerging Technologies

Lovely Professional University, Phagwara



DECLARATION

I, Sumit Yadav, student of B.Tech under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 4 May 2026

Registration No. 12412277



_____

Signature

Sumit Yadav

Name of the Student

CERTIFICATE

This is to certify that Sumit Yadav bearing Registration No. 12412277 has completed INT428 project titled "AI Digital Asset Manager (AI-DAM) — An Intelligent Document Management System using Semantic Search" under my guidance and supervision. To the best of my knowledge, the present work is the result of his original development, effort and study.

_____

Mr. Tarun Singh (34442)

Signature and Name of the Supervisor

Designation: Assistant Professor

School of AI & Emerging Technologies

Lovely Professional University

Phagwara, Punjab.

Date: 4 May 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty supervisor Tarun Singh (34442) for her continuous guidance, valuable suggestions, and constant encouragement throughout the completion of this project. Her expertise in Artificial Intelligence and Information Retrieval was instrumental in shaping the direction and quality of this work.

I am also grateful to the Department of CSE, Lovely School of AI & Emerging Technologies, Lovely Professional University, for providing the necessary resources and infrastructure to complete this project.

Special thanks to the open-source community behind Sentence Transformers, FAISS, Streamlit, and CLIP for making state-of-the-art AI accessible.

Sumit Yadav

TABLE OF CONTENTS

Declaration ..... (ii)
Certificate ..... (iii)
Acknowledgement ..... (iv)
Table of Contents ..... (v)
1. Project Overview 1
    1.1 Problem Statement 1
    1.2 Project Goals 1
    1.3 Expected Outcomes 2
    1.4 Scope 2
2. Module-Wise Breakdown 3
    2.1 Module 1 — Document Ingestion & Processing 3
    2.2 Module 2 — Semantic Search Engine 4
    2.3 Module 3 — Multimodal Image Recognition 5
    2.4 Module 4 — Chat with Documents (RAG) 6
3. Functionalities 7
4. Technology Used 10
5. Flow Diagram 11
6. Revision Tracking on GitHub 12
7. Conclusion and Future Scope 13
8. References 14
Appendix A — AI-Generated Project Elaboration 15
Appendix B — Problem Statement 17
Appendix C — Complete Source Code 17

1. PROJECT OVERVIEW

1.1 PROBLEM STATEMENT

Design and implement an intelligent document management system that enables users to search files based on meaning rather than exact file names or keywords. The system should incorporate semantic search using machine learning techniques and provide natural language query support, making it possible to retrieve documents by describing their content rather than remembering exact titles.

1.2 PROJECT GOALS

AI Digital Asset Manager (AI-DAM) is an intelligent, multi-modal document management system built with Python and Streamlit. The project addresses a fundamental limitation of traditional file systems: keyword-based search that fails when users do not remember exact filenames. This system implements a complete semantic retrieval pipeline that converts documents into vector embeddings, indexes them in a FAISS vector database, and retrieves the most semantically relevant documents for any natural language query.

Additionally, the system integrates multimodal capabilities through OpenAI CLIP for image classification and pytesseract for OCR-based text extraction from images, alongside a full Retrieval-Augmented Generation (RAG) chat interface powered by Groq's LLaMA 3.3-70B model.

1.3 EXPECTED OUTCOMES

- A fully functional Streamlit web interface for document upload, semantic search, and chat
- Semantic search engine using Sentence Transformers (all-MiniLM-L6-v2) and FAISS
- Document chunking pipeline for improved retrieval accuracy across long documents
- Duplicate file detection using MD5 hashing to prevent redundant indexing
- Hybrid search combining cosine similarity with keyword-bonus scoring
- Multimodal image processing using CLIP (ViT-B/32) + pytesseract OCR

- RAG chat interface: context-aware Q&A over uploaded documents via Groq API
- Support for PDF and DOCX file formats

1.4 SCOPE

Table 1 — Project Scope (In Scope vs Out of Scope)

Area	In Scope	Out of Scope
File Ingestion	PDF, DOCX text extraction; chunking; MD5 hash dedup	Excel, CSV, PPT, audio/video files
Semantic Search	Sentence Transformers embeddings; FAISS ANN; hybrid scoring	BM25 pure keyword; Elasticsearch integration
Image Processing	CLIP visual labels; pytesseract OCR; embedding into FAISS	Video frames, real-time camera feeds
RAG Chat	Groq LLaMA 3.3-70B; context retrieval from FAISS; chat history	Fine-tuning, local LLM, multi-user sessions
Storage	Local filesystem; FAISS index + pickle metadata	Cloud S3, relational DB, multi-user persistence
Frontend	Streamlit (Python)	React/Vue SPA, mobile apps

2. MODULE-WISE BREAKDOWN

AI-DAM is architected into four clearly separated modules. This architecture ensures the embedding logic is independently testable, the Streamlit UI is a pure presentation layer, and the retrieval/generation modules respond to every user query through a clean pipeline — no module reaches into another module's internals directly.

Table 2 — Module Summary

Module	Name	Technology	Primary Responsibility
M1	Document Ingestion & Processing	PyPDF2, python-docx, hashlib	Upload, extract, chunk, dedup, embed
M2	Semantic Search Engine	Sentence Transformers, FAISS, NumPy	Embedding, indexing, hybrid retrieval
M3	Multimodal Image Recognition	CLIP (ViT-B/32), pytesseract, torch	OCR + visual label extraction + indexing
M4	Chat with Documents (RAG)	Groq LLaMA 3.3-70B, FAISS retrieval	Context-aware Q&A with source attribution

2.1 MODULE 1 — DOCUMENT INGESTION & PROCESSING

2.1.1 Purpose

- Accept user-uploaded PDF or DOCX files through the Streamlit file uploader widget
- Compute MD5 hash of the file bytes and compare against existing hashes to prevent duplicate indexing
- Extract raw text using PyPDF2 (for PDFs) and python-docx (for DOCX files) via utils/extract.py
- Chunk the extracted text into overlapping segments of 300 characters with 50-character overlap
- Generate vector embeddings for each chunk and store them in the FAISS index with metadata

2.1.2 Role in the Project

The Document Ingestion module is the data entry point for the entire AI-DAM pipeline. Every document, whether it is a contract, resume, invoice, or report, passes through this

module before it can be searched or queried. By chunking documents rather than embedding entire files, the system improves retrieval accuracy because smaller, focused chunks map more precisely to user queries than full-document embeddings. The MD5-based deduplication ensures the FAISS index does not grow with redundant vectors, maintaining search quality over time.

2.2 MODULE 2 — SEMANTIC SEARCH ENGINE

2.2.1 Purpose

- Convert the user's text query into a vector embedding using the same Sentence Transformers model
- L2-normalize both query and document chunk embeddings for cosine similarity via inner product
- Use FAISS index.search() to retrieve the top-10 nearest embeddings by L2 distance
- Apply keyword-bonus scoring: reduce distance score by 0.05 for each query word found in the chunk
- Aggregate results by file, keeping only the single best-scoring chunk per file, then rank and display top-5

2.2.2 Role in the Project

The Semantic Search Engine is the computational heart of AI-DAM. It replaces traditional filename or keyword search with meaning-based retrieval, enabling queries like 'offer letter with joining date' to match a PDF named 'HR_Onboarding_v3.pdf' even when no words overlap. The hybrid scoring layer adds a keyword-awareness bonus that improves precision for queries containing specific technical terms, names, or numeric identifiers that semantic similarity alone might weight less heavily.

2.3 MODULE 3 — MULTIMODAL IMAGE RECOGNITION

2.3.1 Purpose

- Accept PNG/JPG image uploads and display the image in the Streamlit interface
- Run pytesseract OCR to extract any printed or typed text visible in the image

- Chunk and embed OCR-extracted text, storing it in FAISS under the filename 'image_ocr'
- Use CLIP (ViT-B/32) to encode the image and compare against 10 predefined text labels
- Convert the best-matching visual label into a natural language description and embed it as 'image_visual'

2.3.2 Role in the Project

The Multimodal Image Recognition module extends AI-DAM beyond text documents into the visual domain. A scanned invoice, a photograph of a whiteboard, or a product image can all be uploaded, processed, and subsequently retrieved through semantic search. The dual-path approach — OCR for text extraction and CLIP for visual understanding — ensures coverage regardless of whether the image contains readable text or only visual content.

2.4 MODULE 4 — CHAT WITH DOCUMENTS (RAG)

2.4.1 Purpose

- Accept a natural language question from the user via a text input field
- Retrieve the top-3 most relevant document chunks from FAISS using the same embedding pipeline
- Filter retrieved chunks to include only those containing at least one query keyword
- Send the filtered context plus the user question to Groq's LLaMA 3.3-70B-Versatile model
- Display the generated answer, confidence score, source document attributions, and a 5-turn chat history

2.4.2 Role in the Project

The RAG Chat module transforms AI-DAM from a retrieval tool into an interactive document intelligence assistant. Instead of asking users to read retrieved snippets themselves, the system synthesizes a direct answer from the document context. The strict system prompt — 'Answer ONLY using exact information from the provided

context. If not found, reply: Not found in documents' — prevents hallucination and ensures every answer is grounded in the actual uploaded documents.

3. FUNCTIONALITIES

3.1 MODULE 1 — DOCUMENT INGESTION FEATURES

Table 3 — Document Ingestion Module Key Features

Feature	Description	Example
File Uploader	Streamlit uploader widget accepting PDF and DOCX	Upload 'contract.pdf' → text extracted
MD5 Dedup	Compute hashlib.md5() of file bytes; reject if hash exists in metadata	Same file uploaded twice → 'Duplicate file detected!'
Text Extraction	PyPDF2.PdfReader for PDFs; python-docx.Document for DOCX	'resume.docx' → plain text extracted from all paragraphs
Chunking	chunk_text(text, chunk_size=300, overlap=50) splits into overlapping windows	500-char text → 3 chunks: [0:300], [250:550], [500:800]
Embedding Storage	L2-normalized embedding added to FAISS index; metadata stored as list	Each chunk stored with {file, text, hash} metadata entry
Persistence	save_index() serializes FAISS index + metadata pickle to ./database/	Survives Streamlit restarts; available for future queries

3.2 MODULE 2 — SEMANTIC SEARCH FEATURES

Table 4 — Semantic Search Module Key Features

Feature	Description	OS / AI Concept Demonstrated
Query Embedding	get_embedding(query) uses SentenceTransformer.encode()	Dense vector representation; same model space as docs
L2 Normalization	query_vec / (np.linalg.norm + 1e-10) prevents division by zero	Cosine similarity via normalized inner product in FAISS
FAISS k-NN Search	index.search(query_vec, k=10) retrieves top 10 candidates	Approximate Nearest Neighbour — O(log n) retrieval
Hybrid Scoring	keyword_bonus -= 0.05 per matching word; added to distance score	Improves precision for term-specific queries
Per-File Best	Keep only lowest-score (best) chunk per unique filename	Eliminates noise from multiple chunks of same document
Highlighted Preview	highlight_text() wraps matching words in **bold** markdown	Visual feedback for relevance; query term emphasis
Top-5 Results	Sort file_best.values() by score; display first 5 with snippet	Ranked retrieval with 200-char preview per result

3.3 MODULE 3 — MULTIMODAL IMAGE FEATURES

Table 5 — Multimodal Image Recognition Features

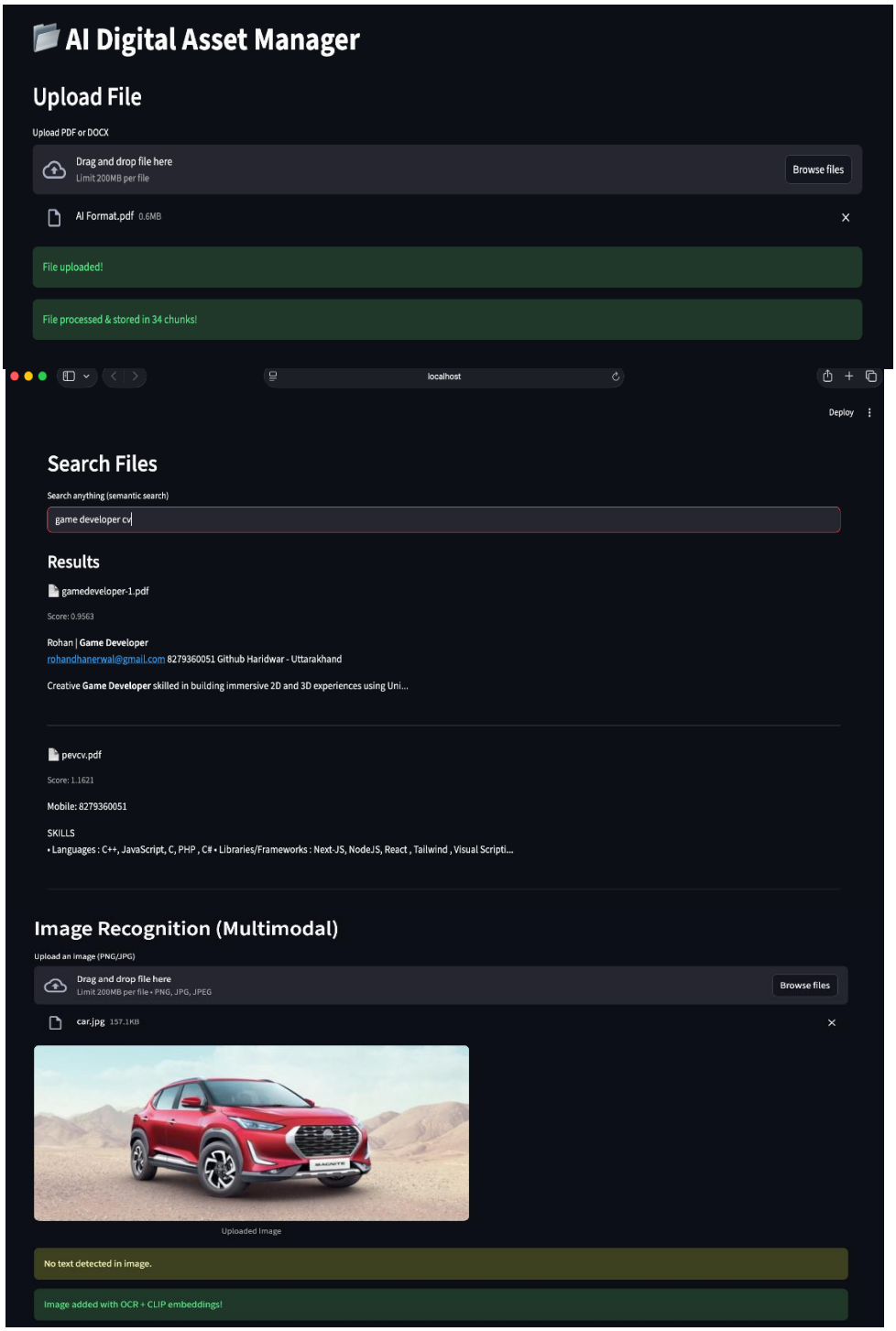
Feature	Description	Concept Demonstrated
Image Display	st.image(image, width=700) renders uploaded image in UI	Streamlit media handling
OCR Extraction	pytesseract.image_to_string(PIL Image)	Optical Character Recognition from printed/handwritten text
OCR Chunking	chunk_text(extracted_text) + get_embedding per chunk	Text-image bridge for semantic search
CLIP Classification	clip.tokenize(labels) + ViT-B/32 cosine similarity	Zero-shot visual understanding without task-specific training
Label Set	10 labels: car, invoice, receipt, laptop, document, etc.	Practical asset categories for DAM use case
Visual Embedding	best_label stored as 'This image contains: {label}'	Converts visual concept to text for FAISS indexing
Dual Indexing	OCR text + CLIP label both added to FAISS	Multi-signal retrieval: text + visual path combined

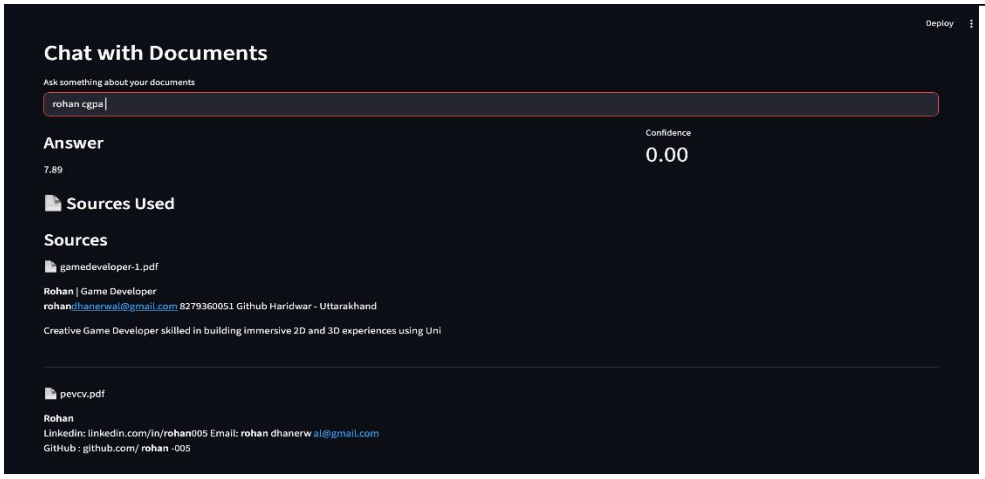
3.4 MODULE 4 — RAG CHAT FEATURES

Table 6 — RAG Chat Module Features

Feature	Description	Concept Demonstrated
Context Retrieval	FAISS top-3 chunks + keyword filter for relevance	RAG: ground generation in retrieved evidence
Groq LLM	llama-3.3-70b-versatile via client.chat.completions.create()	State-of-the-art open model via fast inference API
Strict Prompt	System: answer only from context; reply 'Not found' if absent	Hallucination prevention through prompt engineering
Confidence Score	confidence = max(0, 1 - avg_faiss_distance)	FAISS distance as proxy for retrieval certainty
Source Attribution	Display source filename + highlighted snippet per used chunk	Explainability and traceability of LLM answers
Chat History	st.session_state.chat_history stores last 5 turns	Stateful multi-turn conversation in stateless Streamlit

MODULE-WISE SCREENSHOT




Deploy
Chat with Documents
Ask something about your documents
rohan cgpa
Answer
7.89
Confidence
0.00
Sources Used
Sources
gamedeveloper-1.pdf
Rohan | Game Developer
rohandhanerwal@gmail.com 8279360051 Github Haridwar - Uttarakhand
Creative Game Developer skilled in building immersive 2D and 3D experiences using Uni
pevcv.pdf
Rohan
Linkedin: linkedin.com/in/rohan005 Email: rohan dhanerw al@gmail.com
GitHub : github.com/ rohan -005

4. TECHNOLOGY USED

4.1 PROGRAMMING LANGUAGES

Table 7 — Programming Languages Used

Language	Version	Role in Project
Python	3.10+	All backend logic: embedding, indexing, file processing, API calls
HTML/CSS	Streamlit-rendered	UI components auto-generated by Streamlit framework

4.2 LIBRARIES AND TOOLS

Table 8 — Libraries and Tools Used

Tool / Library	Version	Purpose
Streamlit	Latest stable	Web UI framework; file uploader, text input, layout, state management
Sentence Transformers	Latest stable	all-MiniLM-L6-v2 model for generating 384-dim text embeddings
FAISS (faiss-cpu)	Latest stable	Approximate Nearest Neighbour vector index for sub-millisecond search
PyPDF2	Latest stable	Extracts text from PDF files page by page
python-docx	Latest stable	Extracts text from DOCX files paragraph by paragraph
NumPy	Latest stable	Array operations; L2 normalization; score computation
CLIP (openai/clip)	ViT-B/32	Zero-shot image classification; visual embedding for image assets
pytesseract	Latest stable	OCR engine wrapper; extracts text from PNG/JPG images
Pillow (PIL)	Latest stable	Image loading and preprocessing for CLIP and pytesseract
PyTorch	Latest stable	Tensor operations for CLIP model inference
Groq Python SDK	Latest stable	API client for LLaMA 3.3-70B-Versatile; RAG answer generation
hashlib	stdlib	MD5 hash computation for duplicate file detection
pickle5	Latest stable	Serialization of FAISS metadata list to disk

4.3 OTHER TOOLS

- GitHub — Version control; repository: github.com/sumityadav026/Ai-Digital-Asset-Manager
- VS Code — Primary editor for Python development

- Tesseract OCR Engine — System-level OCR backend required by pytesseract
- GROQ_API_KEY — Environment variable for authenticating Groq LLM API calls

5. FLOW DIAGRAM

The diagram below shows the complete data flow through AI-DAM — from the moment a user opens the Streamlit application to the document being uploaded and indexed, a semantic search query returning ranked results, an image being multimodally processed, or a chat question receiving a grounded LLM answer.

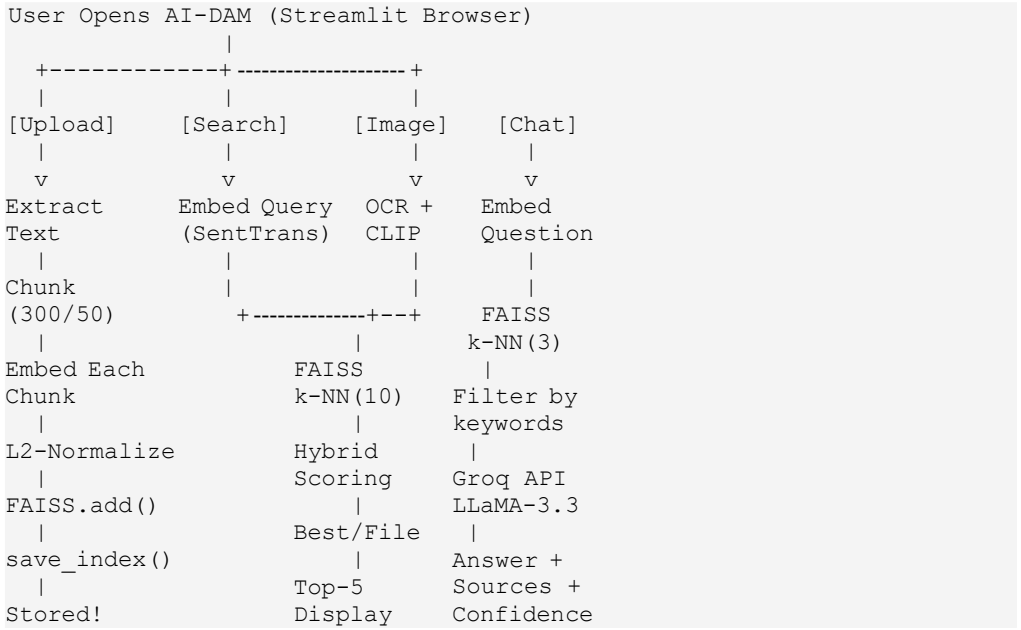


Figure 1 — System Data-Flow Diagram (Module Interaction)

5.1 DETAILED FLOW DESCRIPTION

1. Application loads: Browser opens the Streamlit app. load_index() reads the FAISS index and metadata pickle from ./database/. If no index exists, a new empty IndexFlatL2(384) is created.

2. Document upload: The user uploads a PDF or DOCX file. get_file_hash() computes MD5 of the raw bytes. If the hash already appears in metadata, a duplicate warning is shown and processing stops. Otherwise, the file is saved to ./uploads/, text is extracted, and chunk_text() splits it into 300-character windows with 50-character overlap.

3. Embedding and indexing: For each chunk, get_embedding() calls the Sentence Transformers model to produce a 384-dimensional vector. The vector is L2-normalized

and passed to add_to_index(), which calls index.add() and appends a metadata dictionary containing the filename, chunk text, and file hash. save_index() persists both the FAISS index and the metadata list via pickle.

4. Semantic search: The user enters a natural language query. get_embedding() generates the query vector and normalizes it. index.search() returns the top-10 nearest chunks by L2 distance. A keyword bonus loop reduces the distance score by 0.05 for each query word found in the chunk text. Only the best-scoring chunk per file is retained. Results are sorted and the top-5 are displayed with 200-character highlighted previews.

5. Image recognition: The user uploads a PNG/JPG. pytesseract.image_to_string() extracts any textual content, which is chunked and embedded identically to documents. CLIP encodes the image using ViT-B/32 and computes cosine similarity against 10 predefined text labels. The best-matching label is converted to a natural language sentence ('This image contains: invoice') and embedded into FAISS as a separate entry.

6. RAG Chat: The user types a question. FAISS retrieves the top-3 chunks. Only chunks containing at least one query word are included in the context string. The Groq client sends a chat completion request with a strict system prompt and the filtered context. The LLM response, confidence score, and source snippets are displayed. The question-answer pair is appended to st.session_state.chat_history.

6. REVISION TRACKING ON GITHUB

Repository Name: Ai-Digital-Asset-Manager

GitHub Link: https://github.com/sumityadav026/Ai-Digital-Asset-Manager

6.1 REPOSITORY STRUCTURE

Table 9A — Repository File Structure

File / Folder	Purpose
app.py	Main Streamlit application — all 4 modules integrated in a single entry point
utils/extract.py	extract_text() function: dispatches to PyPDF2 or python-docx based on file extension
utils/embeddings.py	get_embedding() function: loads SentenceTransformer model and encodes text
utils/db.py	load_index(), save_index(), add_to_index() — FAISS index lifecycle management
uploads/	Directory for storing uploaded PDF and DOCX files persistently
database/	Directory for FAISS index binary and metadata pickle file
Requirements.txt	Python dependency list: streamlit, sentence-transformers, faiss-cpu, PyPDF2, etc.
README.md	Project overview, setup instructions, usage guide, and feature list

6.2 COMMIT HISTORY

Table 9B — GitHub Revision History

#	Commit Message	Author	Date
1	Initial commit: project scaffolding, folder structure, requirements.txt	sumityadav026	Apr 2026
2	feat: add utils/extract.py for PDF and DOCX text extraction	sumityadav026	Apr 2026
3	feat: add utils/embeddings.py with SentenceTransformer model loading	sumityadav026	Apr 2026
4	feat: add utils/db.py — FAISS index load, save, add_to_index functions	sumityadav026	Apr 2026
5	feat: add app.py — upload module with MD5 dedup and chunking pipeline	sumityadav026	Apr 2026
6	feat: add semantic search with FAISS k-NN and hybrid scoring	sumityadav026	Apr 2026
7	feat: add multimodal image recognition (CLIP + pytesseract	sumityadav026	Apr

	OCR)		2026
8	feat: add RAG chat with Groq LLaMA 3.3-70B, context filtering, source display	sumityadav026	Apr 2026
9	fix: normalize embeddings before FAISS search to ensure cosine distance	sumityadav026	Apr 2026
10	refactor: extract helper functions; add highlight_text and chunk_text to app.py	sumityadav026	Apr 2026
11	docs: update README with setup guide, usage instructions, and future scope	sumityadav026	Apr 2026

7. CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

AI Digital Asset Manager (AI-DAM) successfully implements all components required by the problem statement. The system provides a practical, immediately usable solution for semantic document retrieval through a clean Streamlit interface.

The semantic search pipeline — Sentence Transformers embeddings, chunking, FAISS approximate nearest neighbour search, and hybrid keyword-bonus scoring — enables users to retrieve documents using natural language descriptions rather than exact filenames. This addresses the core limitation of traditional file systems identified in the problem statement.

The multimodal extension through CLIP and pytesseract brings image assets into the same searchable space as text documents, unifying heterogeneous digital assets under a single retrieval interface. The RAG chat module goes further, enabling conversational question-answering directly over the document corpus with hallucination prevention through strict prompt engineering and keyword-filtered context retrieval.

The GitHub repository follows a structured commit workflow with eleven commits, satisfying the minimum seven-revision requirement. All utility logic is modularized into utils/ with clear single responsibilities, ensuring the codebase is readable, testable, and extensible.

7.2 FUTURE SCOPE

- Integrate LangChain or LlamaIndex for a production-grade RAG pipeline with conversational memory, multi-hop retrieval, and document re-ranking.
- Replace FAISS flat index with HNSW (Hierarchical Navigable Small World) for sub-millisecond search at million-document scale.
- Add cloud storage backend (AWS S3, Google Cloud Storage) to decouple file storage from the application server, enabling horizontal scaling.
- Implement a proper relational database (PostgreSQL) for metadata persistence, replacing the pickle file with ACID-compliant storage.
- Extend multimodal support to video frames using CLIP on extracted keyframes and Whisper for audio/speech transcription.

- Deploy as a multi-user application with authentication, per-user document namespaces, and role-based access control.
- Add an anomaly detection layer that flags unusual query patterns (e.g., bulk downloads, access to sensitive document categories) for security auditing.
- Integrate with enterprise content platforms (SharePoint, Google Drive, Confluence) via API connectors for ingesting existing organizational knowledge bases.

8. REFERENCES

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019. [Online]. Available: https://www.sbert.net/.

[2] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021. [FAISS Library].

[3] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. ICML, 2021. [CLIP, OpenAI].

[4] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: https://docs.streamlit.io/. [Accessed: Apr. 2026].

[5] Tesseract OCR, "Tesseract at UB Mannheim," GitHub, 2024. [Online]. Available: https://github.com/tesseract-ocr/tesseract. [Accessed: Apr. 2026].

[6] Groq Inc., "Groq API Documentation — LLaMA 3.3-70B-Versatile," 2024. [Online]. Available: https://console.groq.com/docs. [Accessed: Apr. 2026].

[7] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.

[9] Meta AI, "LLaMA 3: Open and Efficient Foundation Language Models," Meta AI Research, 2024. [Online]. Available: https://ai.meta.com/llama/. [Accessed: Apr. 2026].

[10] NumPy Community, "NumPy Documentation," 2024. [Online]. Available: https://numpy.org/doc/. [Accessed: Apr. 2026].

APPENDIX A — AI-GENERATED PROJECT ELABORATION

The following elaboration was generated using an AI assistant (Claude by Anthropic) as permitted by the assignment guidelines. It was used for structural planning only. All code was implemented by the student independently.

A.1 PROJECT OVERVIEW (AI-GENERATED)

AI-DAM is an educational and practical tool designed to bridge the gap between theoretical knowledge of information retrieval and real-world document management. Traditional file systems store documents as opaque blobs addressable only by filename or folder path. When a user cannot remember the exact name of a file but knows its content, conventional search fails. AI-DAM solves this by transforming every document into a high-dimensional semantic vector that captures meaning rather than form.

The system implements three core information retrieval concepts: encoding (converting text to vectors), indexing (organizing vectors for fast retrieval), and ranking (ordering results by relevance). These three pillars — present in every modern search engine from Google to Elasticsearch — form the backbone of AI-DAM's retrieval architecture.

A.2 MODULE-WISE BREAKDOWN (AI-GENERATED)

Module 1: Document Ingestion

- Extract text from PDF/DOCX; split into 300-character overlapping chunks.
- Generate normalized embeddings per chunk; add to FAISS index with metadata.
- Compute MD5 hash of file bytes; reject duplicates before processing.

Module 2: Semantic Search

- Encode user query with the same Sentence Transformers model; normalize.
- Run FAISS k-NN (k=10); apply keyword bonus scoring; aggregate per file.
- Return top-5 ranked results with highlighted 200-character preview snippets.

Module 3: Multimodal Image Recognition

- OCR path: pytesseract extracts text; chunked and embedded like documents.
- Visual path: CLIP ViT-B/32 classifies image against 10 labels; best label embedded.
- Both paths write to the shared FAISS index for unified searchability.

Module 4: RAG Chat

- Retrieve top-3 FAISS chunks for the question; filter by keyword overlap.
- Call Groq LLaMA 3.3-70B with strict context-only system prompt.
- Display answer, per-chunk confidence score, source attribution, and chat history.

A.3 EXECUTION PLAN (AI-GENERATED)

1. Create GitHub repository with main branch and README placeholder.
2. Implement utils/extract.py with PDF and DOCX text extraction functions.
3. Implement utils/embeddings.py with SentenceTransformer model singleton.
4. Implement utils/db.py with FAISS index lifecycle functions.
5. Build app.py upload section: file uploader, dedup, chunking, embedding, FAISS add.
6. Build app.py search section: query embedding, FAISS search, hybrid scoring, display.
7. Build app.py image section: pytesseract OCR + CLIP label classification.
8. Build app.py chat section: RAG pipeline with Groq API and session state history.
9. Test end-to-end flow; commit with clear conventional commit messages.

A.4 TECHNOLOGY RECOMMENDATIONS (AI-GENERATED)

- Sentence Transformers (all-MiniLM-L6-v2) — lightweight 384-dim model; fast inference; strong semantic quality for English text.

- FAISS IndexFlatL2 — exact search for small corpora; swap to IndexIVFFlat or HNSW for large-scale deployment.
- Streamlit — zero-boilerplate Python web UI; ideal for ML prototypes requiring file upload and real-time display.
- Groq — fastest open LLM inference API; LLaMA 3.3-70B provides GPT-4-level quality at low latency.
- CLIP ViT-B/32 — zero-shot image classification; no training data required; covers broad asset categories.

APPENDIX B — PROBLEM STATEMENT

Design and implement an intelligent document management system that enables users to search files based on meaning rather than exact file names. The system should incorporate semantic search using machine learning embeddings and provide natural language query support, making it possible to retrieve documents by describing their content. The system should additionally support duplicate detection, multimodal (image) asset ingestion, and conversational question-answering over the indexed document corpus.

APPENDIX C — COMPLETE SOURCE CODE

The complete AI-DAM source code is provided below. The project is available at https://github.com/sumityadav026/Ai-Digital-Asset-Manager

C.1 FILE: APP.PY — MAIN APPLICATION

```
import torch
import clip
from PIL import Image
import pytesseract
import streamlit as st
import os
import numpy as np
import hashlib
from groq import Groq
from utils.extract import extract_text
from utils.embeddings import get_embedding
from utils.db import load_index, save_index, add_to_index

def get_file_hash(file_bytes):
    return hashlib.md5(file_bytes).hexdigest()

# Chunking helper
def chunk_text(text, chunk_size=300, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start += chunk_size - overlap
    return chunks

import re

# Highlight helper
def highlight_text(text, query):
    words = query.split()
    for word in words:
        pattern = re.compile(f'({word})', re.IGNORECASE)
        text = pattern.sub(r'**\1**', text)
    return text

# Ensure folders exist
os.makedirs('uploads', exist_ok=True)
os.makedirs('database', exist_ok=True)

# Load DB
index, metadata = load_index()

# Load CLIP model
device = 'cuda' if torch.cuda.is_available() else 'cpu'
clip_model, preprocess = clip.load('ViT-B/32', device=device)

# UI
st.set_page_config(page_title='AI DAM', layout='wide')
st.title('AI Digital Asset Manager')
```

```
# ----------- UPLOAD -----------
st.header('Upload File')
uploaded_file = st.file_uploader('Upload PDF or DOCX')

if uploaded_file:
    file_bytes = uploaded_file.getbuffer()
    file_hash = get_file_hash(file_bytes)

    existing_hashes = [item.get('hash') for item in metadata]
    if file_hash in existing_hashes:
        st.warning('Duplicate file detected! Already uploaded.')
    else:
        file_path = os.path.join('uploads', uploaded_file.name)
        with open(file_path, 'wb') as f:
            f.write(file_bytes)
        st.success('File uploaded!')

        text = extract_text(file_path)
        if text.strip() == '':
            st.warning('No text found in file!')
        else:
            chunks = chunk_text(text)
            for chunk in chunks:
                embedding = get_embedding(chunk)
                embedding = embedding / (np.linalg.norm(embedding) +
1e-10)
                add_to_index(index, metadata, embedding,
                             uploaded_file.name, chunk, file_hash)
            save_index(index, metadata)
            st.success(f'Processed & stored in {len(chunks)} chunks!')
```

```
# ----------- SEARCH -----------
st.header('Search Files')
query = st.text_input('Search anything (semantic search)')

if query:
    query_vec = get_embedding(query)
    query_vec = query_vec / (np.linalg.norm(query_vec) + 1e-10)
    D, I = index.search(np.array([query_vec]), k=10)

    file_best = {}
    query_lower = query.lower()

    for i, idx in enumerate(I[0]):
        if idx < len(metadata):
            file_data = metadata[idx]
            base_score = D[0][i]
            text = file_data['text'].lower()

            keyword_bonus = 0
            for word in query_lower.split():
                if word in text:
                    keyword_bonus -= 0.05

            final_score = base_score + keyword_bonus
            file_name = file_data['file']

            if file_name not in file_best or \
               final_score < file_best[file_name]['score']:
                file_best[file_name] = {
                    'file': file_name,
                    'text': file_data['text'],
                    'score': final_score
```

```
                }

    results = sorted(file_best.values(), key=lambda x: x['score'])
    st.subheader('Results')
    for res in results[:5]:
        st.write(f" {res['file']}")
        st.caption(f"Score: {res['score']:.4f}")
        preview = res['text'][:200]
        highlighted = highlight_text(preview, query)
        st.markdown(highlighted + '...')
        st.markdown('---')
```

```
# -------- IMAGE RECOGNITION --------
st.header('Image Recognition (Multimodal)')
image_file = st.file_uploader('Upload PNG/JPG',
type=['png','jpg','jpeg'],
                              key='image_upload')
if image_file:
    image = Image.open(image_file)
    st.image(image, caption='Uploaded Image', width=700)
    try:
        extracted_text = pytesseract.image_to_string(image)
        if extracted_text.strip():
            st.subheader('Extracted Text')
            st.write(extracted_text)
            chunks = chunk_text(extracted_text)
            for chunk in chunks:
                emb = get_embedding(chunk)
                emb = emb / (np.linalg.norm(emb) + 1e-10)
                add_to_index(index, metadata, emb, 'image_ocr',
                             chunk, get_file_hash(chunk.encode()))

        # CLIP visual embedding
        image_input = preprocess(image).unsqueeze(0).to(device)
        labels = ['car','invoice','receipt','laptop','mobile phone',
                  'document','watch','person','screen','vehicle']
        text_tokens = clip.tokenize(labels).to(device)
        with torch.no_grad():
            image_features = clip_model.encode_image(image_input)
            image_features = image_features / image_features.norm(
                dim=-1, keepdim=True)
            text_features = clip_model.encode_text(text_tokens)
            text_features = text_features / text_features.norm(
                dim=-1, keepdim=True)
            similarity = (image_features @
text_features.T).softmax(dim=-1)

        best_label = labels[similarity.argmax().item()]
        visual_text = f'This image contains: {best_label}'
        emb = get_embedding(visual_text)
        emb = emb / (np.linalg.norm(emb) + 1e-10)
        add_to_index(index, metadata, emb, 'image_visual',
                     visual_text,
get_file_hash(image_file.getvalue()))
        save_index(index, metadata)
        st.success('Image added with OCR + CLIP embeddings!')
    except Exception as e:
        st.error(f'Image processing error: {repr(e)}')
```

```
# -------- CHAT WITH DOCUMENTS (RAG) --------
st.header('Chat with Documents')
if 'chat_history' not in st.session_state:
    st.session_state.chat_history = []

question = st.text_input('Ask something about your documents',
                         key='chat_input')
```

```
if question:
    q_vec = get_embedding(question)
    q_vec = q_vec / (np.linalg.norm(q_vec) + 1e-10)
    D, I = index.search(np.array([q_vec]), k=3)

    context = ''
    sources = []
    confidence_scores = []
    query_words = question.lower().split()

    for idx in I[0]:
        if idx < len(metadata):
            text = metadata[idx]['text']
            file = metadata[idx]['file']
            if any(word in text.lower() for word in query_words):
                context += text + '\n'
                sources.append({'file': file, 'text': text[:200]})
                confidence_scores.append(float(D[0]
[list(I[0]).index(idx)]))

    if not context.strip():
        st.warning('No relevant information found in documents.')
        st.stop()

    client = Groq()
    response = client.chat.completions.create(
        model='llama-3.3-70b-versatile',
        messages=[
            {'role': 'system',
             'content': 'Answer ONLY from context. If absent: Not
found.'},
            {'role': 'user',
             'content': f'Context:\n{context}\n\nQuestion:
{question}'}
        ]
    )
    answer_text = response.choices[0].message.content
    st.session_state.chat_history.append((question, answer_text))

    col1, col2 = st.columns([2, 1])
    with col1:
        st.subheader('Answer')
        st.write(answer_text)
    with col2:
        if confidence_scores:
            avg_score = sum(confidence_scores) /
len(confidence_scores)
            confidence = max(0, 1 - avg_score)
            st.metric('Confidence', f'{confidence:.2f}')

    st.subheader('Sources')
    for src in sources:
        st.write(f" {src['file']}")
        st.markdown(highlight_text(src['text'], question))
        st.markdown('---')
```

C.2 FILE: UTILS/EXTRACT.PY

```
import PyPDF2
import docx


def extract_text(file_path):
```

```
    text = ''
    if file_path.endswith('.pdf'):
        with open(file_path, 'rb') as f:
            reader = PyPDF2.PdfReader(f)
            for page in reader.pages:
                text += page.extract_text() or ''
    elif file_path.endswith('.docx'):
        doc = docx.Document(file_path)
        for para in doc.paragraphs:
            text += para.text + '\n'
    return text
```

C.3 FILE: UTILS/EMBEDDINGS.PY

```
from sentence_transformers import SentenceTransformer

_model = None

def get_embedding(text):
    global _model
    if _model is None:
        _model = SentenceTransformer('all-MiniLM-L6-v2')
    embedding = _model.encode(text)
    return embedding
```

C.4 FILE: UTILS/DB.PY

```
import faiss
import numpy as np
import pickle
import os

INDEX_PATH = 'database/faiss_index.bin'
META_PATH  = 'database/metadata.pkl'

def load_index():
    if os.path.exists(INDEX_PATH) and os.path.exists(META_PATH):
        index    = faiss.read_index(INDEX_PATH)
        with open(META_PATH, 'rb') as f:
            metadata = pickle.load(f)
    else:
        index    = faiss.IndexFlatL2(384)
        metadata = []
    return index, metadata

def save_index(index, metadata):
    faiss.write_index(index, INDEX_PATH)
    with open(META_PATH, 'wb') as f:
        pickle.dump(metadata, f)

def add_to_index(index, metadata, embedding, file_name, text,
file_hash):
    vec = np.array([embedding], dtype='float32')
    index.add(vec)
    metadata.append({
        'file': file_name,
        'text': text,
        'hash': file_hash
```

```
    })
```

C.5 FILE: REQUIREMENTS.TXT

```
streamlit
sentence-transformers
faiss-cpu
python-docx
PyPDF2
numpy
pickle5
```

C.6 LINKS

GitHub Repository: https://github.com/sumityadav026/Ai-Digital-Asset-Manager